Introducción a Rust II - cargo



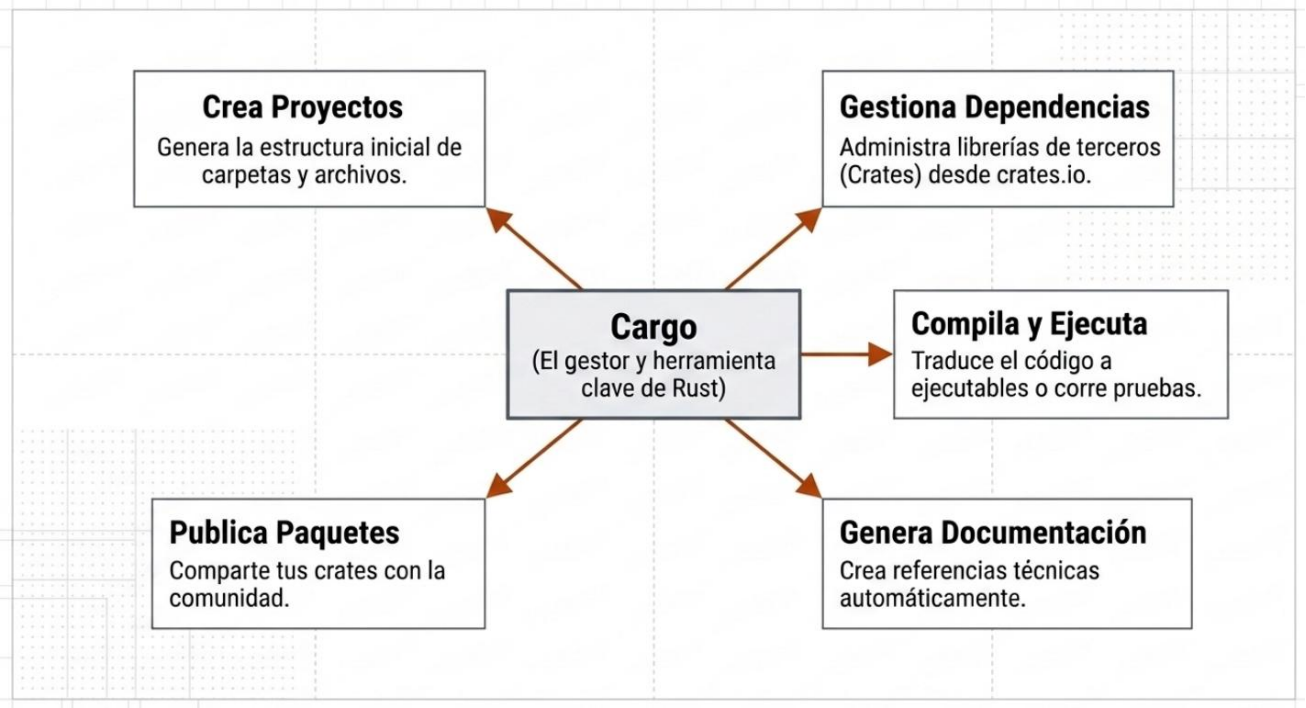
¿Qué es cargo?

Cargo es la herramienta clave de la línea de comandos dentro del ecosistema Rust

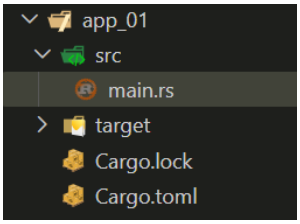
Es una herramienta muy importante que simplifica el proceso del desarrollo.

Escribiendo → `cargo new nombre_proyecto` genera la carpeta y dentro todos los archivos iniciales del proyecto

Características:

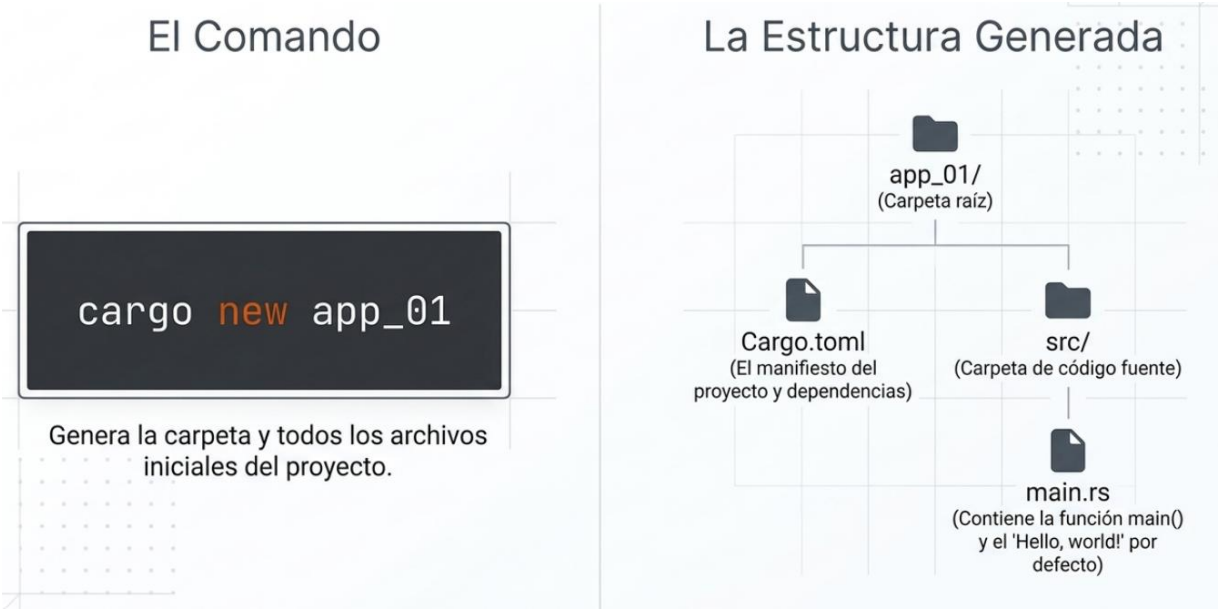


Primeros pasos



Vamos a crear un proyecto desde cero para ver los pasos que hace cargo:

1. Genera la estructura inicial en la terminal con → `cargo new app_01`



2. Borra el contenido del archivo main.rs
3. Escribe manualmente la función main(), el macro println!() Y guarda

```rust
fn main() {
    println!("Hello, world!");
}
```

4. Ejecuta el programa. ▶ Run | ⚙ Debug Esto compila el programa, y con un solo clic… analiza el código fuente y lo traduce a un ejecutable independiente que puede usarse, aunque Rust no esté instalado.

5. Compilar y ejecutar **un archivo** desde el terminal manualmente
    a. Posicionarse en la carpeta del proyecto dentro de src
    b. Compilar el archivo → `rustc main.rs` → esto crea el archivo ejecutable
    c. Ejecutar el programa → `.\main.exe`

6. Compilar y ejecutar el **proyecto completo** desde el terminal manualmente
    a. Posicionarse en la carpeta del proyecto
    b. Compilar todo el código y bibliotecas externas si existen→ cargo build o cargo b

    Se construye la aplicación final con:

    ⇒ **debug** (depurar), es el proceso de encontrar y corregir errores (compilación rápida y no optimizada) que genera un archivo más grande. El ejecutable se guarda en target/debug.

```
PS F:\Cursos\Rust\codigo\app_01> .\target\debug\app_01.exe
Hello, world!
```

    ⇒ **Release,** es el modo para producción, tarda más en compilar porque optimiza el resultado generando un archivo más pequeño que se ejecutará más rápido.
    El ejecutable se guarda en target/release

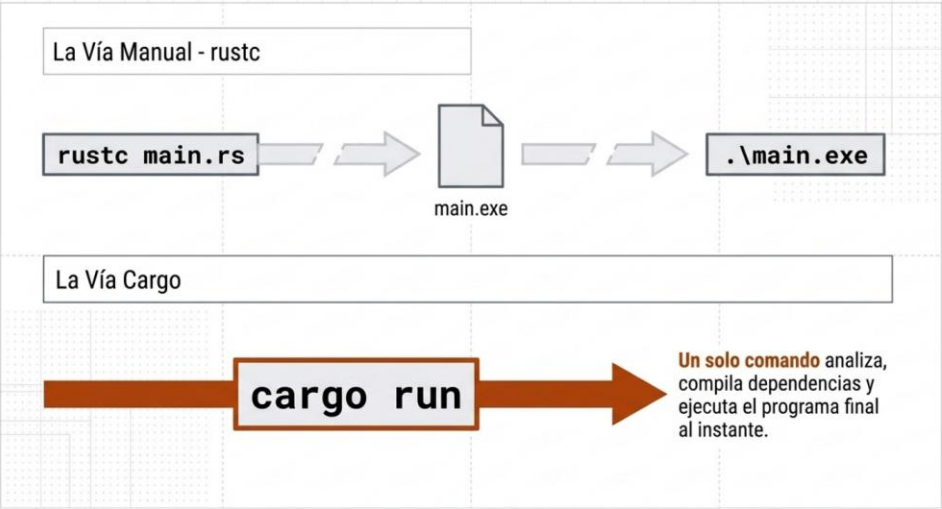
```
cargo  --build --release
```

    c. Limpiar los archivos ejecutables del proyecto si queremos hacer una nueva compilación desde cero. → **cargo clean**

7. Compilar y ejecutar todo el proyecto con un solo comando (modo depuración)
    a. Desde la carpeta superior del proyecto ejecutar el comando → cargo run o r
    b. Si quieres que muestre menos información puedes usar la opción - -quiet o q

8. Comprobar el código antes de compilar es más rápido, no genera un ejecutable.

```
cargo check
```



Parámetros de cargo

cargo new tiene parámetros que cambian opciones en el proyecto generado:

--bin

Usa por defecto el parámetro `--bin` para hacer un **programa binario**. No es necesario ponerlo a no ser que estes usando una configuración donde el tipo predeterminado no es binario.

Cuando hay varios binarios, cargo run no sabe cuál ejecutar y te pide elegir cargo run --bin nombre

⇒ Si los archivos están dentro de la carpeta \bin: No hay que modificar Cargo.toml

```
cargo run --bin prueba
cargo run --bin otro
```

Si los archivos están en otra carpeta: se declara en Cargo.toml

```
[[bin]]
name = "prueba"
path = "src/ejercicios/prueba.rs"
```

Ejecución → cargo run --bin prueba

--lib

Crea un nuevo proyecto de tipo librería en lugar de un ejecutable (binario)

```
cargo new mi_libreria --lib
```

--edition

Especifica la edición de Rust que deseas utilizar para tu proyecto.

- 2024 (por defecto en versiones recientes de Cargo)

```
cargo new mi_proyecto –edition 2018
```

--name

Establece el nombre del paquete. Por defecto, es el mismo que el nombre del directorio.

```
cargo new mi_directorio –name mi_paquete
```

--quiet -q

Minimiza la salida de información en la terminal al crear el proyecto.

```
cargo new mi_proyecto –quiet
```

-v --verbose

Salida verbosa que Incluye salida adicional, como advertencias de dependencia y salida de script de compilación.

--git

Inicializa el proyecto con un repositorio Git y establece la plantilla de inicio desde un repositorio remoto.

```
cargo new mi_proyecto –git https://github.com/usuario/plantilla.git
```

Cuando creas un proyecto con: `cargo new mi_proyecto` Cargo hace automáticamente `git init` y crea:

- Un repositorio Git vacío (carpeta `.git`).
- Un archivo `.gitignore` que ignora la carpeta `target/` (donde van los binarios compilados). Si es una librería, también ignora `Cargo.lock`.

Solo está inicializado: no se hace ningún commit ni se conecta a GitHub u otro servidor remoto. Eso lo haces tú después:

```
git add .
git commit -m "primer commit"
```

Casos en los que NO crea el repositorio

- Si usas `--vcs none`:

```
cargo new mi_proyecto --vcs none
```

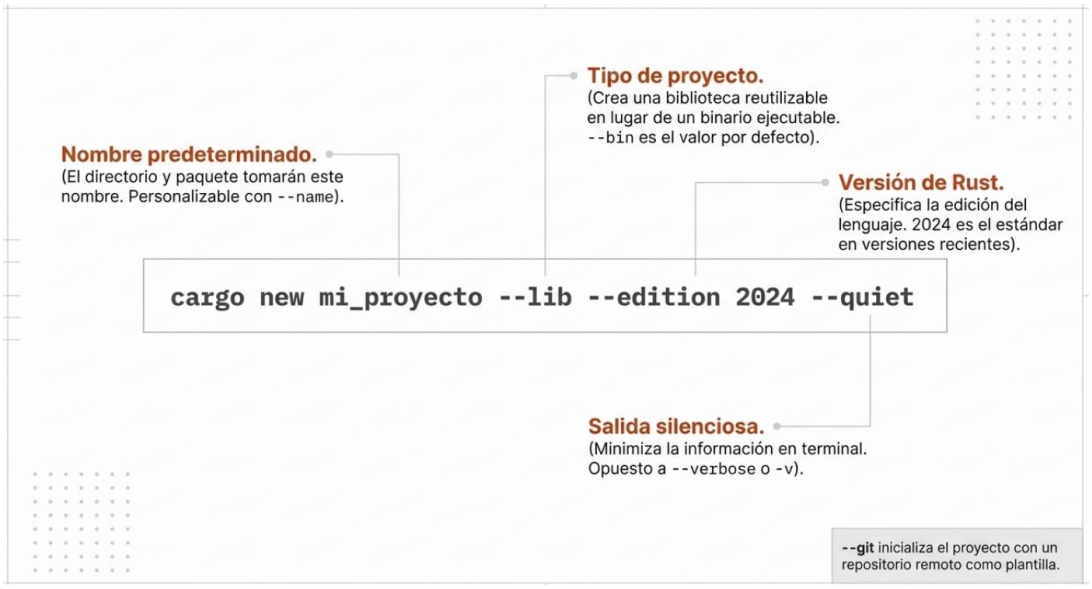
- Si creas el proyecto **dentro de otro repositorio Git ya existente**, Cargo detecta que ya estás en uno y no crea otro.
- Si usas `cargo init` en una carpeta que ya tiene un repo, tampoco lo vuelve a crear.

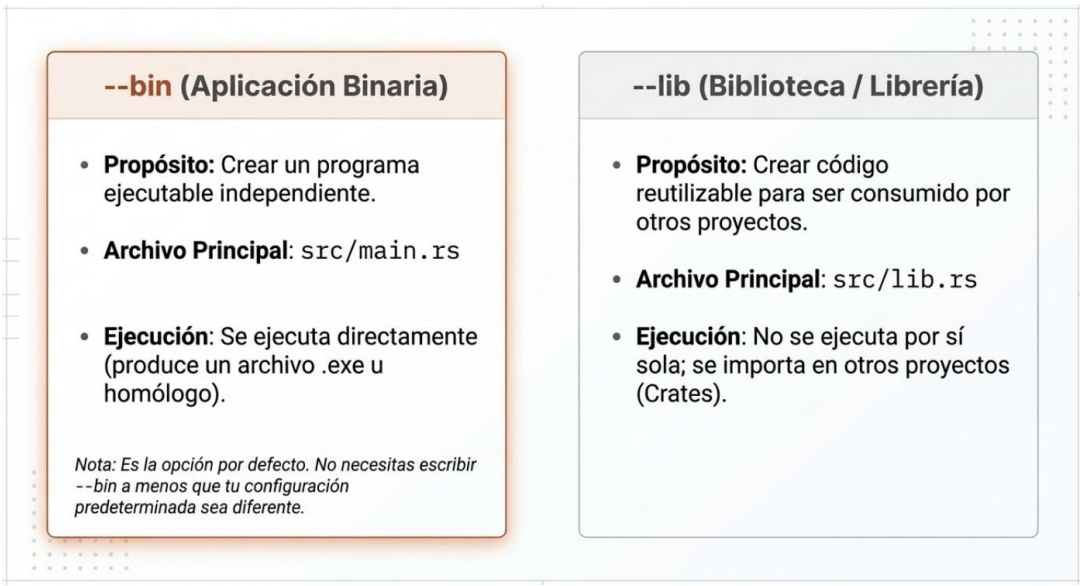
Si quieres cambiar el comportamiento por defecto

Con `--vcs` puedes elegir otro sistema de control de versiones (`git`, `hg`, `pijul`, `fossil`, `none`).

También puedes configurarlo de forma global en `~/.cargo/config.toml`:

```toml
[cargo-new]
vcs = "none"
```





El linter en el mundo de Rust 🦀 : **Clippy**

Un **linter** es una herramienta de software que analiza tu código fuente para buscar errores, malas prácticas, fallos de estilo y posibles bugs antes de que lo compiles o lo ejecutes.

- ✓ **Variables declaradas que nunca usas** (lo que desperdicia memoria).
- ✓ **Errores de sintaxis o de lógica sencillos** (como un paréntesis abierto o un bucle infinito evidente).
- ✓ **Inconsistencias de estilo** (mezclar pestañas y espacios, líneas demasiado largas o nombres de variables mal formateados).
- ✓ **Problemas de seguridad potenciales** (funciones obsoletas o peligrosas).

Mientras que el compilador de Rust (rustc) es increíblemente estricto con la seguridad, **Clippy va un paso más allá y te da consejos de diseño**.

Ejemplo de lo que hace Clippy:
Si escribes un código que funciona perfectamente, pero que podría escribirse de una forma óptima o "idiomática" (más al estilo de Rust), Clippy te mostrará una advertencia en la terminal o en VS Code sugiriéndote el cambio exacto.

Se ejecuta desde la terminal dentro de tu proyecto con: `cargo clippy`

Activar clippy en VSCode

Desde la configuración de Rust-analizer > Check: Command

- Cambiar check por clippy

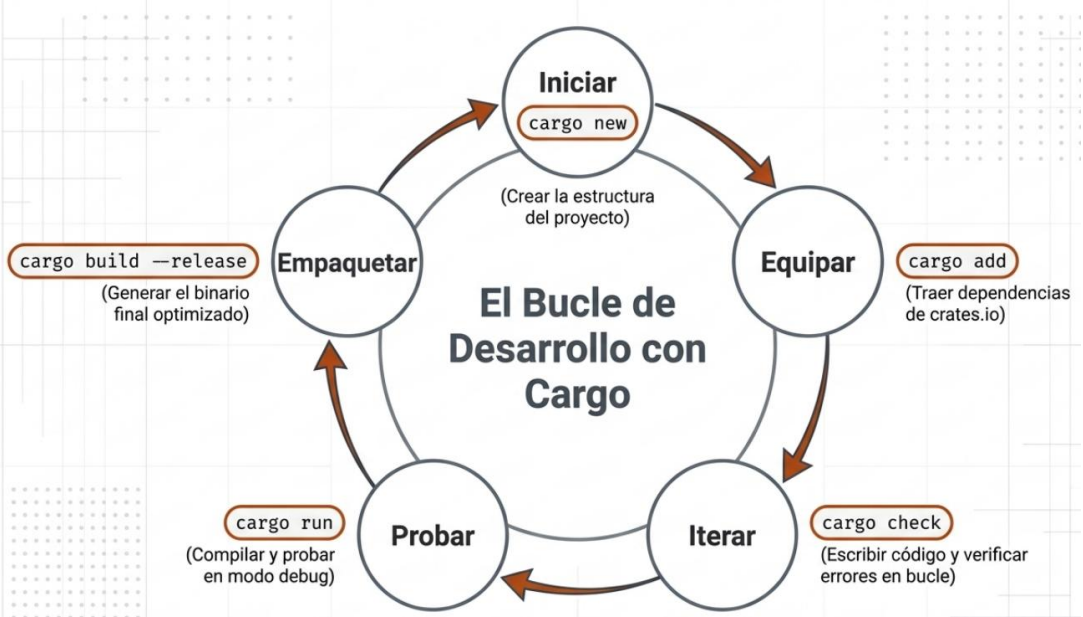


Comandos esenciales

Comando	Qué hace
cargo new mi_app	Crea un proyecto binario (--lib para una biblioteca)
cargo init	Inicializa un proyecto en la carpeta actual
cargo build	Compila en modo debug (--release para optimizado)
cargo run	Compila y ejecuta
cargo check	Comprueba que compila, sin generar binario (rápido)
cargo test	Ejecuta los tests
cargo add depen	Añade una dependencia al cargo.toml
cargo update	Actualiza dependencias dentro de lo que permite Cargo.toml
cargo fmt	Formatea el código
cargo clippy	Linter con sugerencias
cargo doc --open	Genera y abre la documentación
cargo clean	Borra la carpeta target/
cargo publish	Publica el crate en crates.io



Cargo no es dolo una herramienta, es el sistema nervioso que orquesta todo el ciclo de vida de tu software en Rust



Ejemplo dependencias con cargo

Puedes añadir dependencias editando el archivo **cargo.toml** o usando **cargo add.**

Las dependencias se descargan desde crates.io

`cargo add rand` → añade la dependencia rand al proyecto

contenido actualizado de cargo.toml

```
[dependencies]
rand = "0.10.3"
```

```
//use rand::Rng;  //version 0.8.545
use rand::RngExt;  //version 0.10.3
fn main() {
    println!("Aleatorio!");
    let num_aleatorio = rand::rng().random_range(1..=100);
    println!("El numero es: {num_aleatorio}");
}
```

** Los crates se pueden actualizar con el comando → `cargo update`